Taming the Curse of Dimensionality: Quantitative Economics with Deep Learning*

Jesús Fernández-Villaverde
University of Pennsylvania, NBER, CEPR
Galo Nuño
Banco de España, CEPR, CEMFI
Jesse Perla
University of British Columbia

December 5, 2025

Abstract

Deep learning offers a promising avenue for taming the curse of dimensionality in quantitative economics. We begin by examining the unique challenges posed by solving dynamic equilibrium models, particularly the feedback loop between individual agents' decisions and the aggregate consistency conditions required by equilibrium. We then introduce deep learning and illustrate its use by solving the stochastic neoclassical growth model. We conclude with a survey of deep learning applications in economics and offer reasons for cautious optimism.

JEL classification: C61, C63, E27.

Keywords: Deep learning, quantitative economics.

*This paper is based on Jesús Fernández-Villaverde's keynote address at the 2024 Econometric Society Interdisciplinary Frontiers (ESIF) conference on Economics and AI+ML at Cornell. We are very grateful to Francesco Molinari, two referees, Douglas de Araujo, Matthias Rottner, Simon Scheidegger, and Yucheng Yang for their comments, and Bruno Esposito for outstanding research assistance. Jesse Perla gratefully acknowledges support from an SSHRC Insight Grant number 435-2023-0119. The views expressed in this manuscript are those of the authors and do not necessarily represent the views of the Banco de España or the Eurosystem. All remaining errors are ours.

1 Introduction

In this paper, we explore how deep learning tools can help address challenges in quantitative economics that were previously considered intractable. Specifically, deep learning can aid in solving high-dimensional dynamic equilibrium models, which underpin fields as diverse as macroeconomics, industrial organization, international economics, game theory, and finance.

Economists are interested in the "solution" of dynamic equilibrium models (Section 2 will clarify what it means to "solve" a model). This process typically involves finding numerical approximations of unknown functions that appear within economic models, such as value and policy functions, conditional expectations, and distributions of agents. However, these function approximations are hampered by the curse of dimensionality (Bellman, 1957, p. IX), which limits the complexity of dynamic equilibrium models that can be solved to cases with only a few state variables. Deep learning promises that, if properly applied, it can "tame" the curse of dimensionality in many cases.[1]

We aim to make four key points. Our first point is that much of the ongoing revolution in artificial intelligence stems from deep learning, i.e., computer programs that rely on deep neural networks to improve their performance on a task through experience (Murphy, 2022, Section 1.1). While neural networks date back to the 1940s and 1950s (Rosenblatt, 1958), the current surge in interest began in 2012, when Krizhevsky et al. (2012) showed that a deep convolutional neural network (a specific architecture we introduce below) could dramatically outperform existing methods in image recognition.[2]

A series of remarkable achievements followed this breakthrough. Mnih et al. (2015) proposed a reinforcement learning algorithm based on deep neural networks that learned to play Atari video games. Silver et al. (2016) developed another reinforcement learning algorithm that mastered the game of Go, long viewed as too complex for artificial intelligence. Vaswani et al. (2017) introduced the transformer architecture, opening the door to large language models such as ChatGPT. More recently, Trinh et al. (2024) showed how a large language model could be trained to solve mathematical

[1] We pick the term "tame" and not "break" because the curse of dimensionality always shows up in worst-case scenarios. Our goal is to show under what conditions the curse can be managed.

[2] Although some of the algorithms we will use involve subtle elements of learning, it is more accurate to say that we draw on tools from the deep learning toolkit, whether or not a formal learning problem is present.

theorems at the level of the Mathematical Olympiad.[3]

Unsurprisingly, given the success of deep learning in other fields, researchers have begun to explore what it can achieve in economics. The answer has been "a lot," ranging from novel empirical methods to innovative approaches to handling models. In this paper, we focus on the latter, specifically on how deep learning helps economists solve dynamic equilibrium models used across many fields in our discipline.

Our second point is that once the unfamiliar jargon of deep learning is stripped away, the underlying ideas behind it turn out to be natural extensions of tools economists already know well. For example, deep learning function approximations are conceptually close to classical projection methods, such as those based on Chebyshev polynomials. Yet, implementing deep learning at scale differs markedly from conventional approaches. These implementation differences —in optimization, architecture design, regularization, and computational workflow— introduce subtle but important distinctions that matter greatly for how the curse of dimensionality is addressed in practice.

Our third point is that, despite these similarities, economists must remember that, when solving our models, we need to impose equilibrium conditions: the cross-equation restrictions in agents' decision rules that Hansen and Sargent (1980, p. 37) called "the hallmark of rational expectations models." The feedback between agents' optimizing behavior and the consistency requirements of equilibrium generates challenges absent in the typical applications of deep learning in fields such as computer science, the natural sciences, and engineering. While quantitative economists can benefit from deep learning, we must remain mindful that we are computing equilibria (or solving social planner problems) and adapt deep learning tools accordingly.

Our fourth point is that the unreasonable effectiveness of deep learning in economics is not a product of "magic," but follows from intuitive geometrical reasoning. In particular, deep learning geometrically transforms the original state space of the model into an efficient representation that captures the central payoff-relevant features of the states. In both economics and other fields, success in function approximation often depends on identifying the right space for the approximation, not on searching for better approximating functions.

A trivial example illustrates the point. The Cobb-Douglas production function,

[3]Because of tspace limitations, we will focus almost exclusively on deep learning. However, most of our arguments also hold for the broader class of machine-learning tools.

$y = k^{\alpha} l^{1-\alpha}$, links output y to capital k and labor l. One of the first things we explain to students about this function is that it is much easier to work with it in logs, $\log y = \alpha \log k + (1-\alpha) \log l$. Taking logs is simply moving the problem from one space (the levels of k and l) to a more convenient space ($\log k$ and $\log l$) where the function is linear. Deep learning generalizes this idea to the representation of the state variables of arbitrary models. As Tom Sargent loves to say, "[f]inding the state [of a model] is an art." Deep learning helps with this art, as it allows geometric transformations of complex models where our economic understanding of the mathematical structure of the problem does not reach. There is no magic, just better geometry.

Moreover, efficient representations capture features of the underlying state space, leading to good generalization (e.g., off-equilibrium policies for agents consistent with economic intuition). If one can ensure sufficient variation in points within the space of representations, the function approximation may cover an economically significant portion of the original state space.[4]

To illustrate this idea, we will discuss the central role that regularization plays in deep learning. Regularization encompasses all steps the researcher takes to avoid overfitting the data. This can be done explicitly (e.g., by adding a penalty term in the objective function that discourages overly complex models) or implicitly (e.g., through the choice of the optimization algorithm used to "train" the neural network). Thanks to regularization, we obtain good generalization. Without regularization, simply replacing traditional basis functions such as orthogonal polynomials or splines with neural networks does not improve matters much. In short, deep learning succeeds because it provides better geometry, not because it stumbles upon a "clever" basis function for approximation.

Before diving into the main body of the paper, we offer three important caveats. First, our experience is that economists often find the formal probabilistic perspective on deep learning, such as the one followed by Murphy (2022, 2024), easier to relate to their existing methods than perspectives that stress engineering concepts such as circuits or biological analogies such as neurons. Thus, we will avoid jargon about

[4]Economists already use a similar idea in the method by Krusell and Smith (1998), which selects one or a few moments of the distribution of heterogeneous agents to track the evolution of the distribution, or in the oblivious equilibrium solution concept, which solves for best responses to the averages of the behavior of other players (Weintraub et al. 2008; Benkard et al. 2015; Weintraub et al. 2010). Deep learning enables the representation space to be learned and approximated rather than "engineered" by the economist.

"neurons," "activation," and "rectifier units" and stick to the more familiar language of probability, linear algebra, and optimization. Textbooks such as Prince (2023) may provide helpful alternative perspectives to ours. For instance, one promising perspective is to view learning through the lens of compression and complexity.

Second, we must underscore that the interaction between quantitative economics and deep learning is vast. Given the space limitations of this paper, we can only sketch the key ideas. To keep the discussion concise, we will often omit detailed technical specifics. We will provide many references, and we hope the interested reader will be sufficiently intrigued by our exposition to consult them for details. We are tremendously excited about deep learning in quantitative economics and want to convey that sense of discovery without getting bogged down in minutiae.

Third, we will focus exclusively on how quantitative economics and deep learning interact, as we do not have the space to cover the relationship between deep learning and econometrics or finance. Recent related surveys include Athey and Imbens (2019), Scheidegger and Bilionis (2019), Nagel (2021), Kelly and Xiu (2023), de Araujo et al. (2024), and Dell (2024).

The rest of the paper is organized as follows. Section 2 describes what it means to solve a dynamic equilibrium model and why this is challenging. Section 3 introduces neural networks, their training, and the reasons for their strong performance. Section 4 illustrates these ideas with the stochastic neoclassical growth model. Section 5 reviews selected applications in economics. Section 6 highlights four key features of deep learning. Section 7 outlines reasons for cautious optimism, and Section 8 concludes.

2 Solving dynamic equilibrium models

Before delving into deep learning, we must outline what it means to solve a dynamic equilibrium model and where the computational challenges arise. Economists may find much of this section familiar, although some classic ideas appear in ways that offer fresh insights. For computer scientists, our treatment may introduce issues that differ from those encountered in other uses of deep learning. In particular, we explain why the concept of "equilibrium" in economics is emphatically *not* the same as "equilibrium" in other fields (the same word, but with entirely different meanings).

The stochastic growth model. Instead of describing an abstract dynamic equilibrium model, we use the stochastic neoclassical growth model, the backbone of modern macroeconomics, as a pedagogical example to introduce some basic concepts.

In this model, a representative household chooses sequences of consumption $\{c_t\}_{t=0}^{\infty}$ and capital $\{k_{t+1}\}_{t=0}^{\infty}$ to solve:

$$\max_{\{c_t, k_{t+1}\}} \mathbb{E}_0 \sum_{t=0}^{\infty} \beta^t \log(c_t),$$

subject to the budget constraint:

$$c_t + k_{t+1} = w_t + (1 + r_t - \delta)k_t, \qquad \forall t \geq 0, \tag{1}$$

where $\mathbb{E}_0$ denotes the conditional expectation operator at time 0, $0 < \beta < 1$ is the intertemporal discount factor, w_t is the wage paid for the inelastically supplied unit of labor that the household has available, r_t is the rental rate of the capital the household owns, and δ is the depreciation rate.

The representative household is also endowed with an initial capital stock k_0 and must satisfy the transversality condition:

$$\lim_{T \to \infty} \mathbb{E}_0 \left[\beta^T k_{T+1} c_T^{-1} \right] = 0, \tag{2}$$

which rules out explosive asset accumulation or debt. The condition requires that the present value of capital in the very long run cannot be positive (otherwise the household would leave resources unconsumed) nor negative (otherwise it would be consuming more than its available resources). Here, c_T^{-1} is the marginal utility of consumption and, therefore, the appropriate valuation of capital. Together with the initial condition k_0, the transversality condition provides the two boundary conditions that close the household's dynamical system.

A representative firm produces output using capital and the unit of labor supplied by the household with the constant-returns-to-scale technology

$$y_t = \exp(z_t)^{1-\alpha} k_t^{\alpha},$$

with $\alpha \in (0, 1)$, subject to productivity shocks $z_{t+1} = \rho z_t + \sigma \nu_t$, with $\nu_t \sim \mathcal{N}(0, 1)$, a persistence $\rho \in (0, 1)$, and a fixed initital productivity z_0. Since we assume that

input markets are competitive, factor prices equal marginal productivities $r_t = \alpha \exp(z_t)^{1-\alpha} k_t^{\alpha-1}$ and $w_t = (1-\alpha)\exp(z_t)^{1-\alpha} k_t^{\alpha}$.

Finally, the aggregate resource constraint in the economy holds: $y_t = c_t + k_{t+1} - (1-\delta)k_t$, i.e., total output can be used for consumption or for capital accumulation.

We are now ready to define a (rational expectations) equilibrium for this model.

Definition 1 *Given an initial condition (k_0, z_0), an equilibrium consists of stochastic processes $\{c_t^*, k_{t+1}^*, w_t^*, r_t^*\}_{t=0}^{\infty}$ and $\{z_t\}_{t=0}^{\infty}$ such that:*

1. *Given the price sequences $\{w_t^*, r_t^*\}_{t=0}^{\infty}$ and the law of motion for productivity, the representative household chooses $\{c_t^*, k_{t+1}^*\}_{t=0}^{\infty}$ to solve its maximization problem.*

2. *The representative firm hires capital and labor competitively to minimize its costs.*

3. *Goods and factor markets clear:*

$$\begin{aligned} y_t^* &= c_t^* + k_{t+1}^* - (1-\delta)k_t^*, \\ k_t^* &= k_t^{*firm} \\ 1 &= \ell_t^{*firm}. \end{aligned}$$

4. *The household's expectations are consistent with the equilibrium law of motion for productivity and prices.*

This (rational expectations) equilibrium requires only that the representative household and the representative firm optimize, that their actions are mutually consistent, and that markets clear (an accounting identity). Economists are interested in this equilibrium because it is a situation in which agents have no incentive to change their behavior given everyone's actions, and hence it is a self-consistent configuration of the model. Also, note that the equilibrium is a stochastic sequence of variables and that this sequence will, in general, display complicated dynamic patterns.

To characterize this equilibrium, it is convenient to adopt the more compact recursive notation: an arbitrary variable x without a subscript is evaluated at time t, while a primed variable x' denotes the corresponding object at time $t+1$. In this formulation, the dynamics of the model are fully summarized by the two state variables, capital k and productivity z.

To solve its optimization problem, the representative household chooses a policy function for capital accumulation, $k'(k,z)$, that satisfies the first-order condition of the optimization problem of the household, usually known as the Euler equation:

$$1 = \mathbb{E}\left[\beta \frac{c(k,z)}{c(k'(k,z),z')}\left(1-\delta+\alpha \exp(z')^{1-\alpha} k'(k,z)^{\alpha-1}\right)\right], \tag{3}$$

where

$$c(k,z) \equiv \exp(z)^{1-\alpha} k^{\alpha} + (1-\delta)k - k'(k,z),$$

follows from the budget constraint (after replacing prices with marginal productivities), and the expectation $\mathbb{E}$ is taken over z'. One can think of $k'(k,z)$ as the decision rule (also known as the policy function) in a standard dynamic programming problem.

Note that the Euler equation (3) is a functional equation: we seek a decision rule $k'(k,z)$ that makes equation (3) hold for all values of k and z, not just at one point, and that also satisfies the transversality condition (2). The reason is that we care about how agents behave across all possible configurations of the state variables, not only along a single path. To optimize, households need to determine what they would do in alternative paths to the equilibrium sequence. This requirement captures the off-equilibrium behavior mentioned in the introduction and reflects that agents consider counterfactual choices when computing optimal actions. Section 4 will explain how we use deep learning to approximate $k'(k,z)$ with high accuracy on a wide domain of the state space.

Then, given a decision rule $k'(k,z)$, the vector of state variables $X \equiv [k \quad z]^{\top}$ follows the Markov process:

$$X' = \Phi(X,\nu) = \begin{bmatrix} k'(k,z) \\ \rho z + \sigma \nu \end{bmatrix}. \tag{4}$$

This law of motion comes from combining the decision rule and the exogenous productivity process.

Formally, an equilibrium Markov process is a measure over the trajectories $\{X_t\}_{t=0}^{\infty} \sim \mathbb{P}_{X_0}$ generated by $X_{t+1} = \Phi(X_t, \nu_t)$ using the optimal policy function $k'(\cdot)$, the exogenous distribution $\nu_t \sim \mathcal{N}(0,1)$, and the initial condition $X_0 \equiv [k_0 \quad z_0]^{\top}$.

Models as equilibrium Markov processes. There is nothing special about the structure of the model we just presented. For a very general class of dynamic equilibrium models, we can combine the agents' optimal behavior (that is, the equivalent of the Euler equation (3)), budget constraints (the equivalent of equation (1)), technologies, information, and exogenous shocks to generate a measure $\{X_t\}_{t=0}^{\infty} \sim \mathbb{P}_{X_0}$.[5] In fact, we can extend the argument to include behavioral agents (for instance, by limiting the agent's ability to solve the maximization problem) and most non-Markovian cases (which can be rewritten as Markov processes by redefining the state variables).

What "equilibrium" means in this context is simply that we impose several consistency conditions to derive $\{X_t\}_{t=0}^{\infty} \sim \mathbb{P}_{X_0}$: (i) agents maximize (perhaps subject to behavioral constraints), (ii) markets clear, and (iii) the resource constraint is satisfied.

The term "equilibrium" often confuses those outside economics, since it is mistakenly associated with the notion of stability in the natural sciences. In reality, most equilibrium Markov processes involve complex, non-stationary dynamics (including chaotic behavior and limit cycles), idle resources, and other features. Moreover, many (if not most) equilibria of interest are not efficient.[6]

Solving the model. Finding the equilibrium measure $\{X_t\}_{t=0}^{\infty} \sim \mathbb{P}_{X_0}$ among all possible measures over X_t is what we call "solving" the model (i.e., in our example, finding a good approximation to $k'(k, z)$). Once we have the equilibrium Markov process, we can simulate it to evaluate objects of interest, such as moments of endogenous variables, impulse-response functions, demand functions, or counterfactuals.

Unfortunately, except for a few special cases with analytic solutions, economists must determine the equilibrium Markov process numerically. The agents' optimization problem is usually nonlinear and stochastic, and when there are strategic interactions among agents (as in game theory), we need to account for them.

The equilibrium feedback loop. Solving a dynamic equilibrium model is subject to an acute curse of dimensionality because we must solve simultaneously for the

[5] When the economic model is deterministic instead of stochastic, the equilibrium Markov processes is a trivial distribution.

[6] When economists refer to "partial equilibrium," they typically mean that some aspects of the equilibrium Markov processes, such as the interest rate, remain exogenously fixed. When economists describe situations of stability, they usually call them "steady states."

optimization problem of the agents and the equilibrium Markov process, while also accounting for the feedback loop between the two (Sargent, 2025).

To see this, let us return to our example. The representative household treats prices (w_t and r_t) as given when solving its optimization problem, even though these prices are equilibrium outcomes of the household's choices. This is not a contradiction: the representative household stands in for a large number of atomistic households that rationally ignore their individual impact on aggregate prices. In equilibrium, the policy function must generate prices that clear markets, but each household still takes them parametrically (that is, as exogenously given). Suppose any of us decides to save more in 2026. In that case, we take the interest rate in the capital market as fixed because we understand that the aggregate effect of our saving decision is negligible.[7]

We use the capital policy function $k'(k, z)$ and the exogenous process for z to produce sequences of consumption and capital that satisfy the resource constraints of the economy, but those sequences must also induce prices under which the representative household chooses to consume exactly the quantity implied by the equilibrium Markov processes. As soon as we have more than a couple of state variables, these self-consistency requirements become quite difficult to implement.

In comparison, agent-based models are numerically tractable precisely because they avoid this feedback loop and do not impose self-consistency (Wilensky and Rand, 2015).[8] However, most economists are wary of models without self-consistency because they create arbitrage opportunities. Suppose the evolution of the economy is payoff-relevant for the agents. In that case, the agents have incentives to forecast this evolution when solving for their policies rather than simply following a behavioral rule, and to "exploit" agents who fail to do so by trading in financial markets.

Taming the curse of dimensionality. Given the challenge of computing the equilibrium feedback loop of a dynamic equilibrium model, quantitative economists have traditionally resorted to brute force: solving the functional equations in optimality conditions like the Euler (3) over a large grid of the state space and ensuring that they are consistent with equilibrium. The exception is perturbation solutions, which

[7]Without this parametric behavior, the situation becomes even more complex because agents must consider the strategic impact of their actions on others' actions. This concern explains why models in industrial organization, where these strategic interactions matter, are so hard to solve.

[8]Agent-based models are economies in which aggregate outcomes emerge from the interactions of many heterogeneous agents who follow explicitly specified behavioral rules.

only vow accuracy around some point of interest.[9]

Deep learning promises that, in many contexts (although not all), it can tame the curse of dimensionality when finding equilibrium Markov processes, including the equilibrium feedback loop. If so, this could expand the class of dynamic equilibrium models we can handle and the policy-relevant questions we can study. However, explaining why requires several steps. The first is to present deep learning formally.

3 Neural networks

Recall from our example that we want to approximate a policy function $k'(k, z)$ (or its equivalent in another dynamic equilibrium model). In this section, we discuss how neural networks work as function approximators and how they can deliver an excellent approximation $k'_\theta(k, z)$ indexed by weights θ.

Our overview is not intended to replace a comprehensive textbook exposition. Several excellent texts cover the subject in depth (for example, Prince, 2023), and we encourage the reader to consult them. More importantly, the precise definition of neural networks is secondary here because they can be described in several ways. Instead, we want the reader to focus on two key concepts that will guide our discussion.

First, unlike the traditional approach in economics, which approximates an unknown function $f : \mathcal{X} \to \mathbb{R}$ using a flat functional form such as a linear combination of Chebyshev polynomials, neural networks approximate unknown functions through nested structures. For instance, we aim to approximate $f(X) \approx g(\phi(X))$, where $\phi : \mathcal{X} \to \mathcal{Z}$, $g : \mathcal{Z} \to \mathbb{R}$, and $\mathcal{Z}$ is the "representation space."

Second, while $\phi(\cdot)$ and $g(\cdot)$ are jointly found by the same algorithm, we can think of the process as having two stages. The function $\phi(\cdot)$ provides a better geometrical representation of the underlying space $\mathcal{X}$, one that has properties more conducive to downstream tasks (such as the one implemented by $g(\cdot)$) and to generalization (i.e., to be applicable to data never seen before). When the space $\mathcal{Z}$ is Euclidean and endowed with meaningful norms, this transformation is called an "embedding," which may live in a space of lower or higher dimension than $\mathcal{X}$. The function $g(\cdot)$ maps the enhanced representation $\phi(\cdot)$ to an output, and it can be learned or chosen by the researcher.

[9] Similarly, adaptive sparse grids efficiently span the state space by focusing on key regions (Brumm and Scheidegger, 2017). Notably, both perturbation and sparse grids share with deep learning the strategy of mitigating the curse of dimensionality by concentrating on high-probability regions of the model's state space (a point that will be clarified in Section 4).

Let us now unpack all these ideas in detail.

3.1 Neural networks as function approximators

Imagine we want to approximate ("learn") an unknown function $y = f(X)$, where y is a scalar and $X = \{x_0 = 1, x_1, x_2, \ldots, x_N\} \in \mathcal{X}$ is a vector that includes a constant.[10] The x_n's are the features of the data and $\mathcal{X}$ is the feature space. The dimensionality N can be large. Returning to our model from the previous section, the problem is to find $f(X) = k'(X)$ where $X = [k, z]$.[11]

A basic neural network. A single-layer neural network approximates $f(X)$ with

$$y = f(X) \approx g^{NN}(X; \theta) = \theta_0 + \sum_{m=1}^{M} \theta_m \phi(z_m) \tag{5}$$

where $\phi(\cdot)$ is an activation function and $z_m = \sum_{n=0}^{N} \theta_{n,m} x_n$. The $\phi(z_m)$'s are the representations of the data, and M is the width of the model. Also, $\theta = (\theta_0, \ldots, \theta_M, \theta_{0,1}, \ldots, \theta_{N,M})$ are the weights of the network. Popular activation functions include the sigmoidal function $\phi(z) = \frac{1}{1+e^{-z}}$, the hyperbolic tangent $\phi(z) = \tanh(z)$, the rectified linear unit (ReLU) $\phi(z) = \max(0, z)$, and the softplus $\phi(z) = \log(1 + e^z)$. When the activation function is linear, $\phi(z) = \alpha_0 + \alpha_1 z$, the neural network collapses to a linear regression.

Equation (5) is a chain of geometric transformations: it takes the features of the data X, builds M affine transformations of them $z_m = \sum_{n=0}^{N} \theta_{n,m} x_n$, deforms the shape of these affine transformations through an activation function $\phi(\cdot)$ to obtain M representations, and then recombines these representations into another affine transformation $g^{NN}(\cdot)$. In this way, we combine two linear combinations with one (potentially) nonlinear operation. The notation $g^{NN}(X; \theta)$ highlights that the resulting approximation depends on both X and θ. Section 3.2 below explains how to determine these weights. We call this a single-layer neural network because there is only one level of transformation from X into y.

[10] We can accommodate the case where y is a vector using heavier notation. A natural case is when y is a probability distribution.

[11] Other functions we may want to approximate in economics include conditional expectations or the value function that appears in dynamic programming.

Neural networks were named because their structure resembles biological neurons in animal brains. While biological analogies have informed some developments in deep learning, we do not find much value in pursuing them.

A comparison with projection. We can contrast our function approximation with a neural network in equation (5):

$$f(X) \approx g^{NN}(X;\theta) = \theta_0 + \sum_{m=1}^{M} \theta_m \phi \left(\sum_{n=0}^{N} \theta_{n,m} x_n \right), \tag{6}$$

with a function approximation based on a classical projection:

$$f(X) \approx g^{CP}(X;\theta) = \theta_0 + \sum_{m=1}^{M} \theta_m \phi_m(X), \tag{7}$$

where ϕ_m is, for example, a Chebyshev polynomial.

Both approximations are surprisingly similar. As we noted in the introduction, once we set aside the jargon of deep learning, neural networks appear as natural generalizations of methods already familiar to economists. Equation (6) relies on a single activation function $\phi(\cdot)$, whereas equation (7) employs a family of basis functions $\{\phi_m\}$ such as Chebyshev polynomials. The main tradeoff is that the neural-network specification introduces additional weights $\theta_{n,m}$, which are not needed in (7). But even in classical approximations, researchers frequently engineer transformations of the state vector X based on their knowledge of the model. A key advantage of deep learning is that these transformations can be learned automatically, reducing the amount of handcrafted structure required from the researcher.

The more substantive difference lies in how the weights θ are chosen in practice. Deep learning determines them by minimizing the loss function (8) that we will introduce in a few pages, whereas traditional projection methods often rely on collocation or least-squares fitting. This shift in implementation, achieved by minimizing a loss function, has important consequences for performance at scale. We return to this distinction when we discuss training procedures and when we outline several distinctive features of deep learning in Section 6.

A deep neural network. A deep neural network is a composition of $J > 1$ neural networks (or, if one prefers, of $J > 1$ layers):

$$z_m^0 = \theta_{0,m}^0 + \sum_{n=1}^{N} \theta_{n,m}^0 x_n$$

$$g^{NN(1)}\left(X;\theta^1\right) = z_m^1 = \theta_0^1 + \sum_{m=1}^{M^{(1)}} \theta_m^1 \phi^1\left(z_m^0\right)$$

...

$$\begin{aligned} y \approx g^{DL}(X;\theta) &= \theta_0^J + \sum_{m=1}^{M^{(J)}} \theta_m^J \phi^J\left(z_m^{J-1}\right) \\ &= g^{NN(J)}\left(g^{NN(J-1)}\left(...g^{NN(1)}\left(X;\theta^1\right)...;\theta^{(J-1)}\right);\theta^J\right), \end{aligned}$$

where the weight vector $\theta = (\theta^1, \ldots, \theta^J)$ includes the weights in each layer. The widths $M^{(1)}, M^{(2)}, \ldots, M^{(J)}$ and activation functions $\phi^1(\cdot), \phi^2(\cdot), \ldots, \phi^J(\cdot)$ may differ across layers. The hyperparameter J determines the depth of the neural network. If $J = 1$, we have a standard "shallow" neural network, whereas $J > 1$ denotes a deep neural network. Here, "deep" refers to a network that extends across several layers (from the first to the Jth), not to depth in the sense of philosophical sophistication. The superindex DL in $g^{DL}(X;\theta)$ reflects this structure. From now on, we focus on deep neural networks, although all our arguments apply equally to shallow neural networks.

Deep neural networks are sometimes called feedforward networks or fully connected multilayer perceptrons. The term "feedforward" reflects the fact that the composition of neural networks forms a directed acyclic graph. Below, we will discuss alternative neural network architectures.

Like shallow neural networks, deep neural networks are chains of geometric transformations, where we repeatedly apply affine transformations and deform them via activation functions. This approach (as we will formalize in the next section) can capture intricate nonlinear shapes while preserving a linear approximation structure aside from the activation function, thereby keeping computations efficient.

Other architectures. There is a rich set of neural network architectures beyond the densely connected network we have presented. These include, among others,

convolutional neural networks, generalized adversarial networks, and transformers. Later, we will argue that there is a fundamental connection among these different architectures. For now, it is sufficient to note that the choice of architecture should be dictated by the economics of the problem at hand, in particular, by the types of representations it can capture.

Convolutional neural networks are typically employed for image classification. Regular deep neural networks do not perform well on grid-like topologies (e.g., 2-D images) because they are not invariant to shifts. Imagine two pictures of the same object, where the object is shifted in one of them. A densely connected deep neural network often fails to recognize both pictures as equal. Convolutional neural networks avoid this problem by imposing a particular structure on the hidden layers that makes them translation-invariant. In economics, convolutional neural networks have been used to process images (Voth and Yanagizawa-Drott, 2024) or to perform optical character recognition in historical documents (Shen et al., 2021).

Generalized adversarial networks are employed to generate new data (typically images, but potentially any other data type) with the same characteristics as the original training set (Goodfellow et al., 2014). The idea is to have two neural networks: a generator network that produces candidate outputs (e.g., images) and a discriminative network that evaluates whether the candidates are real or generated. Both networks play a minimax game against each other. In economics, generalized adversarial networks have been used, for instance, for estimation (Kaji et al., 2023).

A transformer is a deep learning architecture based on an attention mechanism (Vaswani et al., 2017). Transformers are the basis of large language models such as ChatGPT. The input text is split into n-grams, encoded as "tokens." At each layer, each token is contextualized within a "context window" via an attention mechanism that amplifies the signal of key tokens and attenuates that of less relevant tokens. The applicability of these large language models extends beyond text, and extensions have been developed to address complex topics such as protein folding (Jumper et al., 2021) and theorem proving (Trinh et al., 2024).

3.2 Training the neural network

Training a neural network means selecting θ such that $g^{DL}(X;\theta)$ is as close to $f(X)$ as possible given some relevant metric (for example, the ℓ_2 norm). Later, we will

return to how this can be done for equilibrium Markov processes, since neither y nor X is observable. Instead, they are endogenous equilibrium objects in the model, such as a value or policy function or state variables.

The first step in training a neural network is to specify a loss function that measures the distance between $f(X)$ and $g^{DL}(X;\theta)$ using training data $\mathbf{Y} = \{y_1, \ldots, y_L\}$ and features $X = \{X_1, \ldots, X_L\}$, where $y_l = f(X_l)$. In most deep learning applications, the training data come from the real world. By contrast, when deep learning is used to solve dynamic equilibrium models, the data are generated by simulation or by selecting a grid over the state or sequence space. Thus, the sample size L is limited only by storage and computing time, which offers considerable flexibility.

A natural loss function is the quadratic loss function $\mathcal{E}(\theta; \mathbf{Y}, \widehat{\mathbf{y}})$ on the differences between the data on outcomes and the forecast of the neural network:

$$\theta^* = \arg\min_{\theta} \mathcal{E}(\theta; \mathbf{Y}, \widehat{\mathbf{y}}) = \arg\min_{\theta} \frac{1}{2L} \sum_{l=1}^{L} \left\| y_l - g^{DL}(X_l; \theta) \right\|^2, \tag{8}$$

where $\widehat{y}_l = g^{DL}(X_l; \theta)$ and $\widehat{\mathbf{y}} = \{\widehat{y}_1, \widehat{y}_2, \ldots, \widehat{y}_L\}$. In the context of the stochastic neoclassical growth model, $y_l = 1$ would be the left-hand side of the Euler equation (3) and $g^{DL}(X_l; \theta)$ is the right-hand side of the Euler equation evaluated at the current network decision rule.

The minimization in equation (8) is typically carried out with first-order descent methods, such as `LBFGS`, stochastic gradient descent, minibatch, or `Adam`, and automatic differentiation. The initial θ is set using domain knowledge (the researcher's sense of reasonable starting values) or drawn from a Normal distribution.

Other loss functions may also be used. For instance, one can add regularization terms such as $\lambda \sum_i |\theta_i|$ (LASSO), $\lambda \sum_i \theta_i^2$ (ridge), or a combination $\lambda_1 \sum_i |\theta_i| + \lambda_2 \sum_i \theta_i^2$ (elastic nets). As noted in the introduction, regularization is important for deep learning, though it is often implemented through the optimization algorithm rather than an explicit penalty. We return to this point later.

3.3 Designing the network architecture

So far, we have treated many aspects of the network architecture (e.g., the activation function $\phi(\cdot)$, the number of neurons M, and the number of layers J) as given. In practice, however, one also needs to design them. These choices are known as the

network hyperparameters.

There are several ways to select hyperparameters. One can try different combinations of M, J, and B to minimize the error function $\mathcal{E}(\theta; \mathbf{Y}, \widehat{\mathbf{y}})$. A more formal method is cross-validation, in which we split the sample, train on one part, and evaluate forecasting performance on the other. Finally, one may drop random network links (i.e., set some weights in θ to zero) and verify that performance remains stable, indicating the architecture's robustness.

Evaluating the quality of the architecture is no different from the testing required for any numerical approximation. For instance, when implementing value function iteration, we must decide how many points to include in the grid of state variables and how to space them. Similarly, when we approximate a function with Chebyshev polynomials, we must determine the order of the approximation and how to weigh the residuals to compute the unknown weights. Thus, tuning hyperparameters for a neural network is no more challenging than in other areas of computational economics.

3.4 Why does deep learning work?

Deep learning is not a "black box." Instead, it moves the original function approximation problem, which usually lies in a difficult-to-characterize space, to an equivalent approximation problem in a more convenient space. Before presenting this argument in detail, we need two results that show, at least in principle, that neural networks can deliver the objects we are after: they are universal approximators and can help tame the curse of dimensionality.

The universal approximation theorem. Neural networks are universal approximators: they can approximate any Borel measurable function mapping finite-dimensional spaces arbitrarily well (Hornik et al. 1989; Cybenko 1989). In other words, they can approximate any standard economic function, including those with jumps or non-differentiability. Unfortunately, this result is of limited practical use, as it does not tell us how to implement deep learning in real-life applications.

Breaking the curse of dimensionality. While neural networks are universal approximators, many classical tools (Taylor, Fourier, or Chebyshev series) can also approximate broad classes of functions. What, then, is special about neural networks? A key insight comes from results showing that they can break the curse of dimensionality

(Bach, 2017). A shallow network of width M attains an integrated square error of order $\mathcal{O}(1/M)$. In contrast, series approximations such as Chebyshev projections yield errors of order $\mathcal{O}(1/M^{2/N})$, where N is the dimensionality of the function. Crucially, the network's accuracy does not deteriorate with the number of inputs. Related theorems hold for deep networks, often with tighter bounds (Jacot et al., 2025).

As before, this result is of limited practical use. To train a network properly, one needs good data representations (i.e., the transformation $\phi(x)$) and regularization. As dimensionality increases, it becomes impossible to maintain high accuracy across the entire space of possible data or states, and one can always design worst-case scenarios where it is not possible to generate good representations or achieve regularization.

Deep learning and geometry. The best argument to explain the success of deep learning is that it searches for convenient geometrical representations of high-dimensional manifolds by applying chained geometric transformations to the features of the data. In other words, they search for a good representation space.

We borrow a well-known example from Chollet et al. (2022, Sec. 2.3.6). Imagine that you have crumpled a sheet of paper into a ball. The paper ball is the function you want to approximate in the computer, but doing so, with all the ball's irregularities, is hard. Standard methods such as splines or Chebyshev polynomials fail as soon as we face an irregular ball in four or five dimensions. The geometric transformations in a neural network expand the ball (the affine transformation) and then unfold it (the activation function) until it becomes a plain sheet of paper again. Approximating a sheet of paper (a linear plane) in a computer is straightforward.

Practical advantages and limitations. Deep learning presents other advantages. First, it enables easy parallelization, a major strength given the current ubiquity of graphics processing units (Fernández-Villaverde and Valencia, 2018). Second, hardware such as field-programmable gate arrays can increase performance by two orders of magnitude (Cheela et al., 2022). Neural networks and the related gradient calculations boil down to matrix–vector products, and these architectures can be optimized to store matrices and vectors and to multiply them in parallel. Finally, neural networks are easy to code, and there are outstanding open-source libraries, such as `PyTorch` and `JAX`, that integrate well with scripting languages such as `Python`.

While deep learning can work extremely well, it is not a silver bullet. Real-

world applications involve clear tradeoffs, and training a network often requires understanding the underlying economic problem. To illustrate this, we return to the stochastic neoclassical growth model.

4 Our example again

Although the goal of using deep learning is to extend the dimensionality of the dynamic equilibrium models we can solve, it is helpful to show how they work in our earlier (and low-dimensional) example of the stochastic neoclassical growth model. While we would not expect deep learning to outperform classical approaches in this case, we will see that the solution still takes only 0.54 seconds on a standard laptop using a single CPU.

Approximating the decision rule. Recall that, in our example, we want to approximate the decision rule $k'(k,z)$ with a neural network $k'_\theta(k,z)$ belonging to the architecture $\mathcal{H}(\theta)$, with weights θ. To save notation, we write k'_θ and leave the dependence on (k,z) implicit.

The first step is to define a loss function analogous to equation (8). A natural choice is the squared difference between the left- and right-hand sides of the Euler equation (3) when, instead of the exact decision rule $k'(k,z)$, we use the approximation k'_θ:

$$\mathcal{E}(k'_\theta, X) = \left[1 - \sum_{i=1}^{M} \omega_i \, \beta \frac{c(k,z)}{c(k'_\theta, z'(z,\nu_i))} \left(1 - \delta + \alpha(\exp(z'(z,\nu_i)))^{1-\alpha}(k'_\theta)^{\alpha-1}\right)\right]^2, \quad (9)$$

where $z'(z,\nu_i) = \rho z + \sigma\nu_i$ is a discretization of the productivity shock using M Gaussian quadrature nodes $\{\nu_i, \omega_i\}_{i=1}^M$ for $\nu \sim \mathcal{N}(0,1)$, where ν_i are the quadrature points and ω_i the quadrature weights. In other models, one can construct analogous loss functions from optimality conditions, the Bellman equation, aggregate constraints, expectation operators, and related equilibrium conditions.

Selecting a loss function is also the first step when implementing a classical projection method. The same criteria that researchers follow in that context (numerical convenience, algebraic simplicity, and the ability to capture all relevant equilibrium decisions) should likewise guide the choice of the loss (9). Later, in Section 5, we will

present additional examples of loss functions and illustrate how their construction depends on the structure of the economic problem being solved.

But all these possible loss functions will share a common feature: $\mathcal{E}(k'_\theta, \cdot)$ is itself a function of X. When choosing the weights θ, we must specify a metric over functions: how should we evaluate $\mathcal{E}$ across different values of X?

One possibility is to minimize (9) over a grid for X. The difficulty is that such a grid becomes infeasible in high dimensions. Even a modest grid of ten points per dimension leads to $10^{\dim(X)}$ nodes, an astronomical number if $\dim(X) > 5$.

A more practical alternative is to evaluate (9) only at selected points. For instance, we may minimize the expected loss with respect to the population distribution $\mu^*(X_0, T_{\text{pop}}; k'^*)$ indexed by an initial condition X_0 and a simulation length T_{pop}:

$$k'^* = \arg\min_{k'_\theta \in \mathcal{H}(\theta)} \mathbb{E}_{X \sim \mu^*(X_0, T_{\text{pop}}; k'^*)} \left[\mathcal{E}(k'_\theta, X) \right]. \tag{10}$$

This approach has the advantage of driving the Euler equation as close to zero as possible where the model actually spends its time, as determined by the distribution $\mu^*(X_0, T_{\text{pop}}; k'^*)$. In that way, the algorithm is particularly focused on improving the neural network approximation in regions of economic importance.[12]

If we knew the true $\mu^*(\cdot)$, then a trivial algorithm to minimize equation (10) would be to sample $\mathcal{D}$ points from $\mu^*(\cdot)$ and solve the empirical risk minimization, i.e., the finite-sample version of equation (10):

$$\theta^* = \arg\min_{\theta \in \theta} \frac{1}{|\mathcal{D}|} \sum_{X \in \mathcal{D}} \mathcal{E}(k'_\theta, X).$$

We could also check the generalization properties of k'^* by drawing a different set $\mathcal{D}_{\text{test}}$ from $\mu^*(\cdot)$ and verifying that $\frac{1}{|\mathcal{D}_{\text{test}}|} \sum_{X \in \mathcal{D}_{\text{test}}} \mathcal{E}(k'_{\theta^*}, X)$ is small enough.

The challenge, of course, is that we do not know $\mu^*(\cdot)$ precisely because of the equilibrium feedback loop.

The equilibrium feedback loop revisited. Section 2 emphasized that the equilibrium feedback between agents' optimal decisions (in our case, the representative household) and the equilibrium constraints makes deep learning problems in economics

[12] This also explains why sampling from a simple uniform or normal distribution is a bad idea: most draws would fall in regions of limited economic importance.

fundamentally different from applications in other fields. We now return to this point.

The equilibrium feedback loop arises in equation (10) because the decision rule k'^* appears on the left-hand side as the solution to the arg min, yet it is simultaneously needed to evaluate the expectation on the right-hand side. The expectation is taken with respect to the distribution $\mu^*(\cdot)$, an equilibrium object that itself depends on k'^*. In other words, the weights defining k'^* are chosen by minimizing a loss computed over a sample generated by that very same k'^* —a genuine fixed-point problem.

Sampling from population distribution. A canonical approach to get around the equilibrium feedback loop and find the fixed-point is to iteratively solve an empirical risk minimization version of equation (10) with samples from a misspecified population distribution and periodically regenerate those samples as the decision rule is refined.

That is, given a k'_i, we solve:

$$k'_{i+1} \equiv \arg \min_{k'_\theta \in \mathcal{H}(\theta)} \mathbb{E}_{X \sim \mu^*(X_0, T_{\text{pop}}; k'_i)} \left[\mathcal{E}(k'_\theta, X) \right],$$

and then repeat the process with the new k'_{i+1}. Note, in particular, that we take the expectation with respect to the misspecified population $\mu^*(X_0, T_{\text{pop}}; k'_i)$, not the true $\mu^*(X_0, T_{\text{pop}}; k'^*)$.

In our case, we will start with a decision rule $k'_1(\cdot)$ from the Solow model with a constant savings rate s_{ss}:

$$k'_1(k, z) = s_{ss} \exp(z)^{1-\alpha} k^\alpha + (1 - \delta)k,$$

and generate $\mathcal{D}_1$ from it, although many other initial guesses are possible.[13] For example, one can obtain a solution to the model via a first-order perturbation and use it to generate the first data draw.

More formally, our algorithm is as follows:

1. Solve the empirical risk minimization problem given the current $\mathcal{D}_i$,

$$\theta_{i+1} = \arg \min_{\theta \in \theta} \frac{1}{|\mathcal{D}_i|} \sum_{X \in \mathcal{D}_i} \mathcal{E}(k'_\theta, X).$$

[13]The Solow model is the same as the stochastic neoclassical growth model except that the savings rate of the representative household is exogenously fixed to a constant.

2. Generate a new $\mathcal{D}_{i+1}$ using samples from $\mu^*(X_0, T_{\text{pop}}, k'_{\theta_{i+1}})$.

3. Iterate for a fixed number of steps, until the empirical risk is below a threshold, or check a "validation" set of points outside of $\mathcal{D}_i$.

While this algorithm often works, it has several limitations. First, there is no guarantee that the iteration will converge to the true $\mu^*(\cdot)$ and, consequently, to the exact decision rule. The existing convergence results from deep learning on empirical risk minimization do not apply here because $\mu^*(\cdot)$ is endogenous to the solution of the empirical risk minimization. Second, and relatedly, the algorithm can be sensitive to the choice of the initial decision rule k'_1 used to generate $\mathcal{D}_1$. We must ensure sufficient variation across the state space for the iterations $k'_{\theta_{i+1}}$ to generalize effectively. Third, even if k'_θ is well approximated, the algorithm might still fail to capture the ergodic distribution accurately. Fourth, it is important to avoid oversampling regions of the state space that imply very small Euler equation errors (e.g., where consumption barely changes across periods) but have near-zero probability in $\mu^*(\cdot)$. These points reduce the empirical risk minimization but do not improve generalization.

Parameters, hyperparameters, and optimizer. Before we implement the algorithm above, we need a few preliminaries. First, we must choose values for the parameters of the model $\{\beta, \alpha, \delta, \rho, \sigma\}$. Since our goal is to illustrate how the procedure works, we adopt the most conventional calibration possible: $\beta = 0.99$, $\alpha = 1/3$, $\delta = 0.025$, $\rho = 0.90$, and $\sigma = 0.01$. With these values, capital in the non-stochastic steady state, k_{ss}, is equal to 29.26.[14]

Next, we select the hyperparameters. We use $M = 7$ nodes for quadrature. Our deep neural network has four layers, each with 64 nodes, using the LeakyReLU activation function $\max\{0, x\} + 0.01 * \min\{0, x\}$, and a final softplus layer $\log(1 + e^x)$. This architecture yields 12,737 weights in θ.

We then generate the simulated data used to evaluate the loss function. We set $T_{\text{pop}} = 60$ and simulate 20 different trajectories (k_t, z_t) for $t = 0, \ldots, T_{\text{pop}}$. We purposely keep T_{pop} relatively short: if T_{pop} is too large, the loss may appear small simply

[14] Recall that the steady state is the point where the variables are constant over time. An equilibrium may or may not imply a steady state (or have several). One can rigorously prove that the stochastic neoclassical growth model has a unique steady state and that it converges to it in the absence of productivity shocks. Also, one can prove that the ergodic stochastic dynamics of the model will move around this steady state.

because many observations occur near the ergodic distribution (i.e., the distribution of endogenous variables to which the model settles after some time), where Euler equation errors are naturally reduced, even if the approximation is poor during the transition.

To explore the robustness of the solution with respect to the initial conditions, each trajectory starts from a different initial point. These initial conditions are drawn from $k_0 \sim \mathcal{N}(0.8\, k_{ss},\, 0.1^2)$ and $z_0 \sim \mathcal{N}(0,\, 0.023^2)$ (the ergodic distribution of productivity induced by its law of motion). We choose the mean of the normal distribution for k_0 to be 20% below the steady state so that we can verify that our method captures the transitional dynamics when the economy starts far from its long-run level of capital.

Finally, for optimization, we will run $i = 1, \ldots, 10$ iterations of the LBFGS optimizer to convergence, then regenerate the data. Given the relatively low number of points, there is no need to use sophisticated stochastic optimizers such as `Adam`.

The algorithm runs in approximately 0.43 seconds using only the CPU.[15] We pretrain k'_θ to the same Solow decision rule we used to generate the initial $\mathcal{D}_0$. In particular, we use LBFGS to fit the target decision rule in five iterations.

A reference solution. To evaluate below how good our deep learning solution is, we compute an approximation of the decision rule $k'_{\text{ref}}(k, z)$ with policy function iteration on the Euler equation along a grid of 100 points in k from $0.7 * k_{ss}$ to $1.4 * k_{ss}$ and 31 points in z covering ± 4 standard deviations of the shock process (see Fernández-Villaverde et al., 2016, for a description of policy function iteration). We will call this solution $k'_{\text{ref}}(\cdot)$ where "ref" stands for "reference solution." This reference solution is extremely close to the exact but unknown decision rule for capital.

Results. The left panel of Figure 1 shows the 20 trajectories for $t = 0, \ldots, T_{\text{pop}}$ that we generate from a constant-savings Solow decision rule and that we use to start minimizing the loss function. The right panel shows the final 50 trajectories of the test set that we generate with the final decision rule k'_θ. All 50 final trajectories converge stochastically to the steady-state capital level (recall that there are recurrent productivity shocks along these paths), exactly as theory predicts.

Clearly, Figure 1 shows that the algorithm has converged to a solution, but how good is this solution? The answer is simple: pretty good.

[15] In this implementation, we used `PyTorch` and ran the code on a standard MacBook Air laptop.

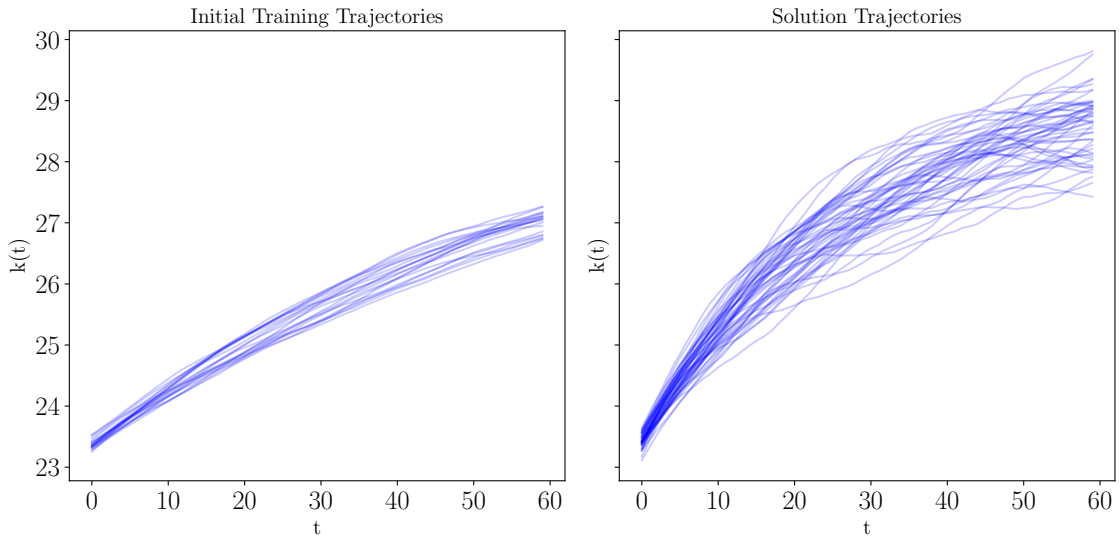

Figure 1: Initial vs. final trajectories of capital.

The left panel of Figure 2 shows the absolute relative policy error for the initial trajectories for $t = 0, \ldots, T_{\text{pop}}$ that we use to minimize the loss function, $\left|\frac{k'(X)-k'_{\text{ref}}(X)}{k'_{\text{ref}}(X)}\right|$, where we employ $k'_{\text{ref}}(\cdot)$ from policy function iteration as the reference solution for benchmarking. The errors range from 0.006 to around 0.001. These errors are not particularly severe because the Solow decision rule, despite its simplicity, is not a poor initial approximation to the reference solution. Nonetheless, these errors are too large for practical use.

The right panel shows the absolute relative policy error for the 50 trajectories of the test set that we generate with the final decision rule k'_θ. The policy errors are now quite low, around 5.00×10^{-5}, which is more than enough for nearly all purposes. They also do not display a significant bias toward small or large t. This performance owes much to the initial dataset, which covered enough regions of the state space for the algorithm to identify the exact decision rule.

Verification of results with test data. To verify that last point, we use the proposed $k'_\theta(\cdot)$ to evaluate an empirical version of the expected loss (10). We sample from $\mu^*(X_0, T_{\text{pop}}; k'_\theta)$ by drawing X_0 and iterating with equation (4) to form a set of points $\mathcal{D}_{\text{test}}$. Using 50 trajectories, we generate a total of 3,000 points. Given these

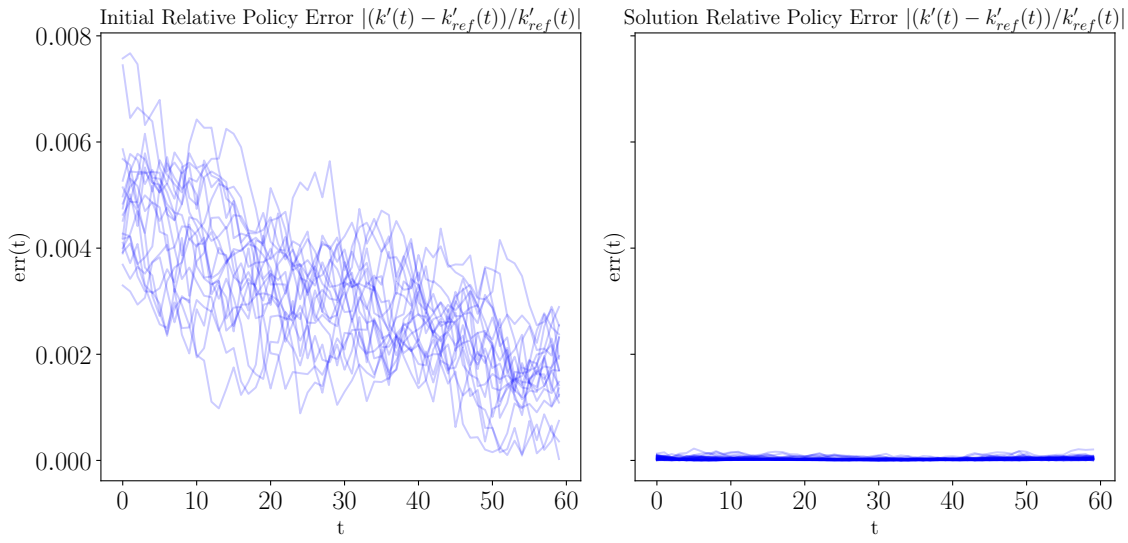

Figure 2: Initial vs. final relative policy errors

samples, the empirical counterpart to equation (10) is:

$$\frac{1}{|\mathcal{D}_{\text{test}}|} \sum_{X \in \mathcal{D}_{\text{test}}} \mathcal{E}(k'_\theta, X).$$

In our simulation, the empirical population risk is 7.74×10^{-9}, which is close to numerical precision. The loss of the empirical risk minimization objective on the final simulated sample is 7.50×10^{-9}.

Because we also have a reference solution $k'_{\text{ref}}(\cdot)$ that we compute with a highly accurate conventional policy function iteration, we can also calculate the relative error:

$$\varepsilon_{\text{rel}} \equiv \frac{1}{|\mathcal{D}_{\text{test}}|} \sum_{X \in \mathcal{D}_{\text{test}}} \left| \frac{k'(X) - k'_{\text{ref}}(X)}{k'_{\text{ref}}(X)} \right|.$$

The absolute relative error of the approximation on $\mathcal{D}_{\text{test}}$ is 2.82×10^{-5}, and the relative error on the final $\mathcal{D}$ is 2.88×10^{-5}. These errors are extremely small?well below one basis point?for both the training and test sets, with the approximation performing slightly better on the latter. This pattern suggests that overfitting is not a major concern.

Finally, we check the transversality condition, which may not hold even when the Euler errors are minimized. To do this, we use 20 simulated trajectories up to $T = 200$

to calculate the expectation in equation (2). See Ebrahimi Kahou et al. (2024) for more details on deep learning and transversality conditions.

Other exercises. Another possible exercise of interest is to estimate the parameters of the model, $\{\beta, \alpha, \delta, \rho, \sigma\}$, and possibly the innovations to the productivity shock, $\{\nu_t\}_{t=0}^{T}$, using observed data from the real world. The general approach is to solve for the equilibrium of the model for a given set of parameters and then match empirical moments or calculate a likelihood that an empirical trajectory would be generated from that equilibrium. See Fernández-Villaverde et al. (2016) for details.

Alternatively, one can directly perturb the process that generates the equilibrium Markov process, a common approach in Bayesian estimation, especially with gradient-based samplers. See Childers et al. (2022), for example. We will return to this point in Subsection 5.4.

5 Applications

Armed with our knowledge of deep learning, we now explore its application to quantitative economics. Due to length constraints, we will be selective in our coverage, leaving out many valuable papers.

5.1 Dynamic programming

As discussed above, traditional numerical techniques for solving dynamic programming problems suffer from the curse of dimensionality. Think, for instance, about solving for the value function of a dynamic programming problem. As the number of dimensions increases, the number of problem elements grows explosively, whether these elements are entries in a high-dimensional table or coefficients in a multidimensional Chebyshev polynomial approximation. In practice, solving dynamic programming problems with more than four dimensions with classical methods becomes extremely costly. Deep learning promises to mitigate this issue and to open the door to larger optimization problems.

Discrete-time problems. First, we consider the case of discrete dynamic programming. We are interested in solving the Euler equation:

$$\mathbb{E}_t \left[h \left(f \left(x_t \right), f \left(x_{t+1} \right) \right) \right] = 0, \tag{11}$$

where $h(\cdot)$ might be nonlinear, x_t is an N-dimensional vector of state variables, and $f(x)$ is a D-dimensional vector of decision rules mapping states into policies.

Maliar et al. (2021) and Azinovic et al. (2022) pioneered approximating the decision rule $f(\cdot)$ with a neural network with weights θ, $\hat{f}(x_t;\theta)$, the same approach used in our earlier application. The network is then trained to minimize the Euler equation error:

$$\min_{\theta} \sum_{l=1}^{L} \left[h\left(\hat{f}\left(x_{l,t};\theta\right), \hat{f}\left(x_{l,t+1};\theta\right)\right)\right]^2,$$

over a set of L simulated samples $\{x_{l,t}\}_{l=1}^{L}$. These samples are typically drawn from the ergodic distribution by simulating the model under the approximated decision rule and the iterative approach described in Section 4. In this way, accuracy improves where the solution actually "lives," which is crucial in high-dimensional spaces where the ergodic distribution occupies only a minute fraction of the state space.

This algorithm is flexible and easy to implement. Variants of it have been used to solve models of saving and consumption (Gorodnichenko et al., 2020), monetary economics (Nuño et al., 2024), climate change (Folini et al., 2024), and asset pricing (Azinovic and Zemlicka, 2024).

Its main shortcoming is the computation of expectations. In low-dimensional settings, expectations can be evaluated with numerical integration, but the task becomes increasingly burdensome as dimensionality grows. Maliar et al. (2021) propose the "all-in-one" expectation operator for efficient Monte Carlo integration. Another alternative is to move to a continuous-time setting (see below).

Duffy and McNelis (2001) and Villa and Valaitis (2024) propose a different approach. Instead of approximating the inner part of the expectation operator, they approximate the operator itself, in the spirit of the parameterized expectations algorithm introduced by den Haan and Marcet (1990). As discussed by Maliar and Maliar (2003), the main drawback is that if the assumed decision rule is far from the true solution, the algorithm is likely to diverge.

Continuous-time models. The difficulty of handling the expectation in equation (11) suggests moving the dynamic optimization problem to continuous time, where the Hamilton-Jacobi-Bellman equation can be evaluated without computing expectations.

For instance, suppose that states x_t follow a diffusion process:

$$dx_t = \mu\left(x_t, c_t\right) dt + \sigma\left(x_t\right) dz_t,$$

where $\mu(\cdot)$ is the drift, $\sigma(\cdot)$ the diffusion, c_t the policies, and z_t a Brownian motion.

If the agent has an objective function $\int_0^\infty e^{-\rho t} u(c_t, x_t)\, dt$, the associated Hamilton–Jacobi–Bellman equation is:

$$\rho V(x) = \max_c \; u(c,x) + \sum_{n=1}^{N} \mu_n(x,c)\, \partial_{x_n} V(x) + \frac{1}{2} \sum_{n_1,n_2=1}^{N} \Sigma_{n_1,n_2}(x)\, \partial^2_{x_{n_1},x_{n_2}} V(x), \quad (12)$$

where $V(\cdot)$ is the value function that encodes the return to the agent of following the optimal decision rule. Here $\Sigma(x) = \sigma(x)\sigma(x)^\top$ denotes the instantaneous covariance matrix of the state process.

No expectations are needed in the Hamilton-Jacobi-Bellman equation because, by Itô's lemma, the last term captures all the uncertainty from the diffusion process. The structure is similar if the state also features Poisson jumps.

The approach in Fernández-Villaverde et al. (2020), Gopalakrishna (2021), Sauzet (2021), and Duarte et al. (2024) is to approximate the value function with a neural network $\widehat{V}(x;\theta)$ and train it to minimize the Hamilton-Jacobi-Bellman error:

$$\min_\theta \sum_{l=1}^{L} \Big[-\rho \widehat{V}(x_l) + u(c(x_l), x_l) + \sum_{n=1}^{N} \mu_n(x_l, c(x_l)) \partial_{x_n} \widehat{V}(x_l) + \sum_{n_1,n_2=1}^{N} \frac{\sigma^2_{x_{n_1},x_{n_2}}(x_l)}{2}\, \partial^2_{x_{n_1},x_{n_2}} \widehat{V}(x_l) \Big]^2.$$

As before, these samples are typically drawn from the model's ergodic distribution by simulating the economy under the approximated policy function or by using a fast, approximate solution such as a first-order perturbation.

The decision rule $c(\cdot)$ can be derived from the first-order condition

$$\partial_c u(c,x) + \sum_{n=1}^{N} \partial_c \mu_n(x,c)\, \partial_{x_n} \widehat{V}(x) = 0,$$

or approximated by a second neural network trained to minimize the resulting first-order condition error.[16]

A further advantage of continuous time is that computing partial derivatives of the

[16] There is an extensive literature on using deep learning to solve partial differential equations, including but not limited to Hamilton-Jacobi-Bellman equations. See, for example, Han et al. (2018) and Sirignano and Spiliopoulos (2018).

value function is straightforward. One important detail, however, is that the neural network must incorporate the boundary conditions in the loss function.

Huang (2023) follows a different route. Instead of working with the Hamilton-Jacobi-Bellman equation, he formulates the problem as a system of forward-backward stochastic differential equations. He still approximates the value function with a neural network, but evaluates the loss function along simulated paths of the value process.

5.2 Reinforcement learning

Reinforcement learning is a rapidly expanding field in computer science and robotics (Murphy, 2025). It studies dynamic programming problems in which the transition law of the states is unknown and must be learned from how states evolve under chosen actions. Many of the breakthroughs mentioned in the introduction rely on deep reinforcement learning, where deep neural networks approximate value and policy functions (e.g., Silver et al., 2016).

Reinforcement learning distinguishes between "model-free" and "model-based" methods. Model-free approaches, such as Q-learning or deep policy gradients, update value or policy functions directly from observed state–action–reward sequences without attempting to recover the underlying transition law. Model-based approaches instead estimate or specify the transition dynamics and use them to solve the dynamic program, typically by simulating the system forward.

This distinction parallels familiar economic concepts. Model-based reinforcement learning resembles rational expectations equilibria: agents (or algorithms) work with an explicit law of motion for the state and choose optimal actions consistent with that law. Model-free reinforcement learning is closer to adaptive or least-squares learning rules (e.g., Evans and Honkapohja, 2001), where agents update beliefs from realized outcomes without recovering the structural transition law.

Several recent papers apply these ideas in economics. Covarrubias (2023) uses deep reinforcement learning to study dynamic oligopolistic interactions and their implications for monetary policy transmission. Atashbar and Shi (2022) solve a real business cycle model using deep reinforcement learning. Chen et al. (2023) apply similar methods to a monetary economy, and Hinterlang and Tänzer (2021) use deep reinforcement learning to compute optimal monetary policy. Broader surveys of reinforcement learning in economics and finance are provided by Charpentier et al.

(2020) and Atashbar and Shi (2022).

5.3 Heterogeneous-agent models

The stochastic neoclassical growth model we have used so far has only one representative household. However, economists are very interested in solving models with heterogeneous agents, that is, agents who differ in non-trivial ways. For instance, we study models with income and wealth inequality among households as well as differences in age, education, marital status, and other characteristics. And we consider models with many different and heterogeneous firms.

A canonical example of a model with heterogeneous agents, which we will discuss below, is the model by Krusell and Smith (1998). The authors take the stochastic neoclassical growth model we introduced earlier, but instead of assuming a single representative household, they assume a continuum of atomistic (i.e., measure-zero) households that differ in their (stochastic) labor income. Because insurance markets are incomplete and risky labor income cannot be fully hedged, their consumption and wealth evolve differently. Thus, in models with heterogeneous agents, we need to distinguish between the idiosyncratic states of the agents (e.g., the wealth of agent $i \in [0, 1]$ at time t) and the aggregate states (e.g., total wealth at time t, aggregate productivity, and similar variables).

More importantly, the distribution of idiosyncratic states G_t (e.g., the distribution of wealth among all agents at time t) is itself a state variable. The reason is that this distribution matters for equilibrium outcomes. The evolution of the interest rate in an economy with aggregate wealth 10, divided equally between two agents (the wealth of agent 1 at time t is five and the wealth of agent 2 is five), will differ from the evolution of the interest rate in an economy with the same aggregate wealth but divided unevenly (the wealth of agent 1 at time t is six and the wealth of agent 2 is four). Agents, therefore, need to carry G_t as part of their state variables because G_t determines prices.

Since G_t is a distribution, it will belong in general to an infinite-dimensional space. Furthermore, the operator $H(\cdot)$ that governs its evolution,

$$G_{t+1} = H(G_t, S_t),$$

given the other aggregate states of the economy S_t, is itself a non-standard operator

that cannot be handled with standard finite-dimensional calculus.

Again, we encounter the equilibrium loop: $H(\cdot)$ is an equilibrium object because the distribution G_t is determined jointly by agents' optimal decisions and by the equilibrium Markov process. Agents' choices depend on G_t through prices and other aggregates, and G_{t+1} reflects those choices. This mutual dependence generates the feedback between individual decision-making and the law of motion of the distribution. For example, in the model of wealth heterogeneity by Krusell and Smith (1998), next period's distribution depends on current wealth, optimal savings, aggregate prices, and shocks.

Existing methods. There is a plethora of computational methods to solve models with heterogeneous agents under perfect-foresight conditions (i.e., without stochastic shocks) or subject to aggregate shocks up to a first-order approximation (e.g., Ahn et al., 2018, Boppart et al., 2018, Winberry, 2018, Auclert et al., 2021) and even higher orders (Bhandari et al., 2023, and Bilal, 2023).

The problem becomes much more challenging when we try to solve a heterogeneous-agent model with aggregate shocks over the entire state space, which is essential to study, for example, the role of nonlinearities in these models.

Krusell and Smith (1998) propose a bounded-rationality solution for a model with a continuum of atomistic agents. When making decisions, these agents summarize G_t with a few moments and forecast the law of motion of these moments. For example, the agents can summarize the distribution of wealth of the households by just (the log of) its mean, $\log \widehat{k}_t$.

An equilibrium in this model is a straightforward extension of our definition of equilibrium in Section 2, except that we add the requirement that the perceived law of motion for $\log \widehat{k}_t$ matches the simulated law of motion for $\log \widehat{k}_t$ along the equilibrium path (we skip the details because the notation becomes heavy). The question is how to build the perceived law of motion.

Krusell and Smith (1998) show that a (log-)linear perceived law of motion

$$\log \widehat{k}_{t+1} = a_0(S) + a_1(s) \log \widehat{k}_t$$

where $a_0(S)$ and $a_1(S)$ are coefficients that depend on the aggregate exogenous state variables (e.g., aggregate productivity), offers a good approximation to the exact

actual law of motion of their model. The reason is that, for their parameterization, the exact actual law of motion is very close to linear.[17]

However, there is no guarantee that a linear perceived law of motion will also approximate models with more pronounced nonlinearities at the macroeconomic level. Once we leave the linear world, the set of possible functional forms becomes infinite. Paraphrasing Tolstoy: "Linear models are all alike; every nonlinear model is nonlinear in its own way."

Approximating the perceived law of motion. Fernández-Villaverde et al. (2023) propose a solution to this problem in continuous time by approximating the perceived law of motion with a neural network trained on simulated data (Fernández-Villaverde et al., 2024, extend the idea to discrete time).

More concretely, one can extract a finite number of features from G_t. These can be moments, Q-quantiles, mixture-of-normals weights, or many other options. One then stacks the features of the distribution in a vector μ_t and assumes that μ_t follows a perceived law of motion:

$$\mu_{t+1} = h(\mu_t, S_t),$$

instead of the exact $H(G_t, S_t)$.

We can parameterize $h(\mu_t, S_t)$ as $h(\mu_t, S_t; \theta)$ using a neural network, where θ is the vector of weights. We then choose θ to ensure that an economy in which μ_t follows $h(\mu_t, S_t; \theta)$ replicates as closely as possible the behavior of an economy in which G_t follows $H(\cdot)$. To do so:

1. Guess an initial value θ^0 for the network weights and set $n = 0$.

2. Construct a time series of μ_t by simulating from the perceived law of motion and the relevant equilibrium conditions of the model.

3. Train the network on the simulated data and obtain θ^{n+1} by minimizing a quadratic error function between the realized μ_{t+1} and the network?s prediction $h(\mu_t, S_t; \theta^n)$.

4. Iterate on steps 2–3 until $||\theta^{n+1} - \theta^n|| < \epsilon$ for a given norm $||\cdot||$ and tolerance level $\epsilon > 0$.

[17] See Moll (2025) for the argument that rational expectations are implausible in heterogeneous-agent settings, given the infinite-dimensional forecasting they require, and for proposed alternatives such as temporary equilibrium, survey expectations, least-squares learning, and reinforcement learning.

This algorithm nests the approach of Krusell and Smith (1998) as a special case, since the linear formulation of the perceived law of motion used by Krusell and Smith (1998) is a simple neural network.

There is also related work by Gu et al. (2023) in continuous time, as well as its applications to search-and-matching models (Payne et al., 2024) and asset pricing (Gopalakrishna et al., 2024).

DeepHAM. One shortcoming of the Fernández-Villaverde et al. (2023) approach is that the researcher must select the relevant moments $h(\mu_t, S_t)$ based on the economics of the problem. Instead, Han et al. (2022) propose approximating the state distribution by a set of optimal generalized moments. Neural networks represent these generalized moments, which are determined automatically by the algorithm, a method they call DeepHAM. Deep neural networks also approximate the value and policy functions, and the objective is optimized over directly simulated paths, as in the methods discussed in the previous subsection.

Interestingly, Han et al. (2022) choose an objective function based on cumulated utility rather than first-order conditions, which makes DeepHAM useful for solving models with complex first-order conditions, e.g., constrained efficiency problems such as those in Dávila et al. (2012). This choice also makes DeepHAM resemble model-based reinforcement learning.

Exploiting symmetry. Ebrahimi Kahou et al. (2021) propose a method for solving models with a large but finite number of heterogeneous agents using deep learning. This setting differs from the problems described so far, which involve a continuum of atomistic agents. However, both are closely related since traditional heterogeneous-agent models can be viewed as the limit of a large, finite population. The authors tame the curse of dimensionality by exploiting symmetry in the perceived law of motion to engineer an efficient representation of the state space.

5.4 Estimation

Deep learning can also be used to estimate dynamic equilibrium models more efficiently. This is a computationally intensive task that requires solving the model for a large number of parameter values and then evaluating the likelihood using a sequential Monte Carlo filter (Fernández-Villaverde and Guerrón-Quintana, 2021).

Kase et al. (2024) propose a method to accelerate estimation in heterogeneous-agent New Keynesian models with aggregate shocks. First, they compute the equilibrium using deep learning as proposed by Maliar et al. (2021) and discussed in Section 5.1.

Second, they treat the parameters as pseudo-state variables by exploiting the scalability of deep learning. Then, they train this extended neural network with the parameters as inputs over the entire parameter space. This step yields a mapping from the parameter space to the model's equilibrium law of motion, rather than just a solution at a single point in the parameter space. Although treating the model's parameters as pseudo-state variables makes training more challenging, the computational gains are substantial compared to repeatedly evaluating the model's solution mapping at as many points as needed to obtain an appreciable evaluation of the likelihood function.[18]

Third, they train an additional neural network that provides a direct mapping from the model parameters to the value of the likelihood function. For this strategy, they evaluate the likelihood using the particle filter for different parameter combinations over the parameter space and use these as data points to train the second neural network. Because neural networks have strong generalization properties, this training requires only thousands of data points, significantly reducing computation time compared to conventional estimation, which typically requires millions of draws.

A related idea appears in Friedl et al. (2023), who solve the model as a function of its states and parameters in a single step using deep learning. Instead of simulating the moments, however, the authors simulate the social cost of carbon and then compute the Sobol indices, univariate effects, and Shapley values for global uncertainty quantification.

Finally, Naubert (2025) develops a Bayesian method for estimating globally solved nonlinear models, using a mixture density network for initial states, and shows that this improves parameter recovery relative to steady-state initialization.

6 Four features of deep learning

In Section 3, we noted a key tradeoff between the parsimony of basis functions and the richer parameterization of $\theta_{n,m}$ in equation (6) relative to equation (7). This insight is part of a broader pattern. To clarify further the comparison between classical solution

[18] A similar approach, dubbed "deep surrogates," is employed in finance by Chen et al. (2021).

methods for dynamic equilibrium models and neural networks, it is useful to outline four distinguishing features of deep learning.

Distinguishing feature 1: Overparameterization and regularization. Classic function approximations, such as equation (7), typically restrict the number of weights θ to be smaller than the amount of data to avoid overfitting. In contrast, deep learning often employs models with orders of magnitude more weights than the amount of data.

However, simply counting weights is a poor metric for assessing an approximation or its relationship to the data (e.g., Curth et al., 2023). Computer scientists and statisticians instead refer to the flexibility of the functional form as "capacity," a notion that generalizes weight counting and aligns better with concepts of complexity.

From the perspective of quantitative economics, overparameterized approximations with high capacity allow us to fit intricate functions that arise in very high-dimensional state spaces of many dynamic equilibrium models. In particular, we can construct solutions that match the model structure at all grid points to numerical precision, thereby avoiding the usual bias-variance tradeoff.

Nonetheless, as argued in the introduction, avoiding this tradeoff depends on using regularization to guide algorithms toward solutions with desirable properties. In deep learning, regularization often arises in more subtle ways, such as through the network architecture, the choice of optimizer, or data noise (see Barrett and Dherin, 2020, Smith et al., 2021, and Chiang et al., 2022).[19]

Following our notation from a few pages ago, regularization ensures that we do not identify spurious features in the transformation of the state through $\phi(X)$ and that we avoid overfitting the output through $g(\cdot)$.

At a deeper level, highly overparameterized models introduce an intrinsic form of regularization through the phenomenon known as "double descent." This refers to the shape of the function-approximation error (for instance, the loss function in equation (8)) as a function of model complexity:

1. Error first decreases as the number of weights increases, and the model learns the data ("first descent").

[19]The fact that different combinations of weight values can deliver the same value of equation (8) is not a concern: all such combinations generate the same approximation function. One must avoid overfacile analogies with econometrics, where point identification is often a key desirable property. For solving dynamic equilibrium models, point identification of the θ's is irrelevant.

2. Error then increases once the model becomes flexible enough to memorize the data and overfits.

3. After crossing the interpolation threshold, further increases in the number of weights reduce the error again ("second descent").

Although the double descent pattern has been documented at least since Vallet et al. (1989), it was not widely recognized until Belkin et al. (2019). See, for a general review, Schaeffer et al. (2023) and Wilson (2025), and, in econometrics, Spiess et al. (2023) provide recent examples of double descent arising in causal inference.

Distinguishing feature 2: Representations. A common element of deep learning is the transformation of the underlying data into features that are better suited to the task at hand. This can be done with classical methods using artful "engineering" by the economist (for instance, manually taking first differences), but, as discussed before, in deep learning, it typically means the algorithm learns the transformation on its own. By exploiting patterns in the structure of the data, the network discovers a representation that captures the payoff-relevant features of the dynamic equilibrium model.

Distinguishing feature 3: Concentration of measure. Deep learning distributes approximation errors unevenly across the state space, concentrating accuracy on the "typical sets" of the state variables, which become increasingly representative (and occupy less relative volume) due to concentration-of-measure phenomena (Ledoux, 2001).[20]

In some dynamic equilibrium models, high-dimensional states deliver a blessing of dimensionality rather than a curse: typical sets shrink rapidly because of concentration of measure (one can view this concentration as a law of large numbers for endogenous equilibrium objects).

The key change in perspective is that high dimensionality, whether in the state or parameter space, is not inherently problematic. Classical methods struggle unless one can predetermine the typical set of the state variables with prior knowledge of the model. In contrast, in many deep learning settings, dimensionality is an advantage.

[20]Loosely, the typical set is the region most relevant for computing objects such as conditional expectations. Its geometry is often counterintuitive, especially in high dimensions; see Fernández-Villaverde and Guerrón-Quintana (2021) for an introduction.

More data help estimation, and moving to a continuum limit in mean-field games or to large numbers of discrete agents can simplify equilibrium models.

In general, dimensionality is neither something to fear nor something that must always be "conquered," and solutions are problem dependent. As long as we are willing to ignore the worst case and define success probabilistically, the curse of dimensionality is not a fundamental barrier.

Distinguishing feature 4: High-dimensional optimization. Deep learning leverages advances in software to compute high-dimensional gradients of arbitrary functions and advances in hardware to implement them efficiently. These developments make it feasible to optimize the loss functions associated with dynamic equilibrium models. By contrast, this software and hardware are much less suited for classical methods such as Chebyshev polynomials.

7 Reasons for cautious optimism

Throughout the paper, we have emphasized that the success of deep learning stems from changing the geometry of the representation space. Our goal now is to explain why this shift opens the door to three reasons for cautious optimism.

7.1 Low-dimensional representations

Several methods in economics have successfully handled some high-dimensional problems. First, perturbation works because (i) it focuses on a small region of the state space (i.e., the neighborhood of the deterministic steady state), and (ii) it imposes stability so that solutions do not overfit. Second, we have solutions in dynamic IO —such as the self-confirming equilibria in Fudenberg and Levine (1993) and the related experience-based equilibria in Fershtman and Pakes (2012)— that restrict attention to empirically relevant regions of the state space and impose less structure on off-equilibrium beliefs. Third, Weintraub et al. (2008) address the curse of dimensionality by transforming the state space into a low-dimensional variable that captures the long-run dynamics.

The results in Ebrahimi Kahou et al. (2021) and Ebrahimi Kahou et al. (2024) suggest that deep learning replicates the logic behind why these methods work. For

example, following the exploitation of symmetry in Ebrahimi Kahou et al. (2021), we conjecture that deep learning methods can learn the low-dimensional representations that the three approaches above implicitly rely on. The authors also show that the problems can be solved with fewer ad-hoc restrictions on off-equilibrium beliefs and best responses. Finally, Ebrahimi Kahou et al. (2024) discuss the close connection between regularization in deep learning and the stability that emerges from the inductive bias built into neural networks.

The tradeoff behind deep learning is that some regions of the state space (which, ideally, the equilibrium Markov process reaches only with low probability for a given counterfactual) will be poorly approximated. When nearly the entire state space matters for accuracy, the curse of dimensionality becomes unavoidable.

More generally, economists are trained to treat high dimensionality with caution, and they are rightly skeptical of claims that it can be overcome easily. Yet deep learning transforms the original state into an engineered or learned representation, so the relevant dimensionality is that of the representation rather than that of the raw space. For instance, if the function to approximate is a nonlinear mapping of the mean of the data (as in Krusell and Smith, 1998), the mean itself becomes the ideal representation.

7.2 The manifold hypothesis

The "manifold hypothesis" conjectures that real-world data concentrate on a low-dimensional manifold (Fefferman et al., 2016). In essence, the world is much simpler than it appears, though what counts as "simple" depends on the task at hand. This hypothesis emerges from the interaction between concentration of measure —where a well-behaved high-dimensional stochastic process places most of its probability mass in a small region— and representations that compress the payoff-relevant information in a high-dimensional state into a much lower-dimensional one.

The manifold hypothesis helps explain the remarkable success of deep learning in complex domains such as computer vision and natural language processing. From this perspective, the intrinsic structure of real-world images or text is far simpler than their ambient dimension suggests.

If the manifold hypothesis is plausible for empirical data, it should apply even more strongly to dynamic equilibrium models, which are deliberately constructed

to be simple. The economist may not know the most useful representation of the model, but a small set of payoff-relevant forces governs its underlying structure. Deep learning provides a way to uncover these hidden low-dimensional representations.

This geometric interpretation naturally ties in with work in high-dimensional geometry and machine learning. The manifold hypothesis posits that many high-dimensional datasets lie on low-dimensional manifolds embedded in the high-dimensional spaces where they are encoded (Cayton, 2005). Under this view, deep learning only needs to fit these low-dimensional manifolds within its potential input space. A related line of research explores deeper links between geometry and deep learning by connecting neural network architectures to underlying symmetries (Bronstein et al., 2021).[21] These perspectives suggest that, regardless of architectural details, all neural networks share a common geometric structure, a promising direction that may generate new insights in the coming years.

7.3 Transferability between models and invariants

Deep learning can help us reuse calculations across versions of a dynamic equilibrium model, such as when estimating parameters or performing comparative statics. This transferability arises from using good representations to approximate the state space, since much of the work in fitting a neural network involves finding an effective $\phi(X)$.

Once such a representation is in place, only the final "layer" in the nested function approximation needs to be adjusted to suit a new task. Components of the approximation can then serve as an initial condition or remain fixed while the rest of the model is resolved. This is convenient with deep learning, which allows for "model surgery" (the dissection of pre-trained layers to fix some nested approximations while leaving others to adapt). This process is known as "transfer learning" (Zhuang et al., 2020).

More formally, we can reuse a $\phi(\cdot)$ function from a prior problem and combine it with a new function $g(\cdot)$ to form $f(X) \approx g(\phi(X))$. Once a stable representation $\phi(X)$ is learned and possibly frozen, it still sits inside the computational graph, so modern autodiff tools can propagate derivatives through $g(\cdot)$, through $\phi(\cdot)$, and through the fixed-point solver. This connection makes it practical, when employing differentiable programming, to compute how an entire equilibrium Markov process changes with

[21] Vershynin (2018) provides an introduction to high-dimensional geometry as applied to data, emphasizing that our geometric intuition from three-dimensional Euclidean space is a poor guide in high-dimensional settings.

respect to a parameter. For an introduction to differentiable programming, see Blondel and Roulet (2024), and for an estimation example, see Childers et al. (2022).

8 A glimpse into the future

We are only beginning to see the full impact of the artificial intelligence revolution sparked by Krizhevsky et al. (2012). Nevertheless, its effects are already pervasive in daily life, from self-driving cars taking us to school in the morning to large language models drafting emails to the dean's office. Quantitative economics is no exception.

For decades, economists have worked under significant constraints on the models they could study, often focusing on models they could solve rather than those they wanted to solve. Deep learning, when applied judiciously to tame the curse of dimensionality, can expand the scope of our work in ways that were hard to imagine a few years ago. However, this progress is not due to "magic" but to better geometric representations of the state space. Much work remains to be done to understand representation theory, related ideas such as the double descent phenomenon, and their implications for economics.

These advances hold transformative implications for economics and economic policy. For example, Fernández-Villaverde et al. (2025) review how deep learning could reshape the models used to study the macroeconomic effects of climate change, while Carvalho et al. (2024) employ deep learning to analyze the nonlinear propagation of sectoral shocks. Moreover, the shift in quantitative economics toward microdata-driven models often requires high-dimensional state spaces, making deep learning particularly well-suited to these environments. To echo Howard Carter's famous response on November 26, 1922, when asked by Lord Carnarvon if he could see anything as he peered into King Tutankhamun's tomb, we can also answer about the future of quantitative economics: "Yes, wonderful things!"

References

Ahn, S., G. Kaplan, B. Moll, T. Winberry, and C. Wolf (2018): "When inequality matters for macro and macro matters for inequality," *NBER Macroeconomics Annual*, 32, 1–75.

Atashbar, T. and R. A. Shi (2022): "Deep reinforcement learning: Emerging trends in macroeconomics and future prospects," Working Paper 2022/259, IMF.

Athey, S. and G. W. Imbens (2019): "Machine learning methods that economists should know about," *Annual Review of Economics*, 11, 685–725.

Auclert, A., B. Bardóczy, M. Rognlie, and L. Straub (2021): "Using the sequence-space Jacobian to solve and estimate heterogeneous-agent models," *Econometrica*, 89, 2375–2408.

Azinovic, M., L. Gaegauf, and S. Scheidegger (2022): "Deep equilibrium nets," *International Economic Review*, 63, 1471–1525.

Azinovic, M. and J. Zemlicka (2024): "Intergenerational consequences of rare disasters," Manuscript.

Bach, F. (2017): "Breaking the curse of dimensionality with convex neural networks," *Journal of Machine Learning Research*, 18, 629–681.

Barrett, D. G. and B. Dherin (2020): "Implicit gradient regularization," Tech. Rep. 2009.11162, arXiv.

Belkin, M., D. Hsu, S. Ma, and S. Mandal (2019): "Reconciling modern machine-learning practice and the classical bias–variance trade-off," *Proceedings of the National Academy of Sciences of the United States of America*, 116, 15849–15854.

Bellman, R. (1957): *Dynamic Programming*, Princeton University Press.

Benkard, C. L., P. Jeziorski, and G. Y. Weintraub (2015): "Oblivious equilibrium for concentrated industries," *RAND Journal of Economics*, 46, 671–708.

Bhandari, A., T. Bourany, D. Evans, and M. Golosov (2023): "A perturbational approach for approximating heterogeneous agent models," NBER Working Papers 31744, National Bureau of Economic Research.

Bilal, A. (2023): "Solving heterogeneous agent models with the master equation," NBER Working Papers 31103, National Bureau of Economic Research.

Blondel, M. and V. Roulet (2024): "The elements of differentiable programming," Tech. Rep. 2403.14606, arXiv.

Boppart, T., P. Krusell, and K. Mitman (2018): "Exploiting MIT shocks in heterogeneous-agent economies: The impulse response as a numerical derivative," *Journal of Economic Dynamics and Control*, 89, 68–92.

Bronstein, M. M., J. Bruna, T. Cohen, and P. Veličković (2021): "Geometric deep learning: Grids, groups, graphs, geodesics, and gauges," Tech. Rep. 2104.13478, arXiv.

Brumm, J. and S. Scheidegger (2017): "Using adaptive sparse grids to solve high-dimensional dynamic models," *Econometrica*, 85, 1575–1612.

Carvalho, V., M. Covarrubias, and G. Nuño (2024): "Nonlinearities and amplification in dynamic production networks," Manuscript.

Cayton, L. (2005): "Algorithms for manifold learning," Technical report, University of California at San Diego.

Charpentier, A., R. Elie, and C. Remlinger (2020): "Reinforcement learning in economics and finance," Tech. Rep. 2003.10014, arXiv.

Cheela, B., A. DeHon, J. Fernández-Villaverde, and A. Peri (2022): "Programming FPGAs for economics: An introduction to electrical engineering economics," Working Paper 29936, National Bureau of Economic Research.

Chen, H., A. Didisheim, and S. Scheidegger (2021): "Deep surrogates for finance: With an application to option pricing," Available at SSRN 3782722.

Chen, M., A. Joseph, M. Kumhof, X. Pan, and X. Zhou (2023): "Deep reinforcement learning in a monetary model," Tech. Rep. 2104.09368, arXiv.

Chiang, P.-y., R. Ni, D. Y. Miller, A. Bansal, J. Geiping, M. Goldblum, and T. Goldstein (2022): "Loss landscapes are all you need: Neural network generalization can be explained without the implicit bias of gradient descent," in *The Eleventh International Conference on Learning Representations*.

Childers, D., J. Fernández-Villaverde, J. Perla, C. Rackauckas, and P. Wu (2022): "Differentiable state space models and Hamiltonian Monte Carlo estimation," Working Paper 30573, National Bureau of Economic Research.

Chollet, F., T. Kalinowski, and J. Allaire (2022): *Deep Learning with R*, Manning Publications, 2nd ed.

Covarrubias, M. (2023): "Dynamic oligopoly and monetary policy: A deep reinforcement learning approach," Manuscript.

Curth, A., A. Jeffares, and M. van der Schaar (2023): "A u-turn on double descent: Rethinking parameter counting in statistical learning," Tech. Rep. 2310.18988, arXiv.

Cybenko, G. (1989): "Approximation by superpositions of a sigmoidal function," *Mathematics of Control, Signals and Systems*, 2, 303–314.

Dávila, J., J. H. Hong, P. Krusell, and J.-V. Ríos-Rull (2012): "Constrained efficiency in the neoclassical growth model with uninsurable idiosyncratic shocks," *Econometrica*, 80, 2431–2467.

de Araujo, D. K. G., S. Doerr, L. Gambacorta, and B. Tissot (2024): "Artificial intelligence in central banking," BIS Bulletins 84, Bank for International Settlements.

Dell, M. (2024): "Deep learning for economists," *Journal of Economic Literature*, forthcoming.

den Haan, W. J. and A. Marcet (1990): "Solving the stochastic growth model by parameterizing expectations," *Journal of Business & Economic Statistics*, 8, 31–34.

Duarte, V., D. Duarte, and D. Silva (2024): "Machine learning for continuous-time finance," *Review of Financial Studies*, 11, 3217–3271.

Duffy, J. and P. D. McNelis (2001): "Approximating and simulating the stochastic growth model: Parameterized expectations, neural networks, and the genetic algorithm," *Journal of Economic Dynamics and Control*, 25, 1273–1303.

Ebrahimi Kahou, M., J. Fernández-Villaverde, S. Gomez-Cardona, J. Perla, and J. Rosa (2024): "Spooky boundaries at a distance: Inductive bias, dynamic models, and behavioral macro," Working Paper 32850, National Bureau of Economic Research.

Ebrahimi Kahou, M., J. Fernández-Villaverde, J. Perla, and A. Sood (2021): "Exploiting symmetry in high-dimensional dynamic programming," Working Paper 28981, National Bureau of Economic Research.

Evans, G. W. and S. Honkapohja (2001): *Learning and Expectations in Macroeconomics*, Princeton University Press.

Fefferman, C., S. Mitter, and H. Narayanan (2016): "Testing the manifold hypothesis," *Journal of the American Mathematical Society*, 29, 983–1049.

Fernández-Villaverde, J., K. Gillingham, and S. Scheidegger (2025): "Climate change through the lens of macroeconomic modeling," *Annual Review of Economics*, Forthcoming.

Fernández-Villaverde, J. and P. A. Guerrón-Quintana (2021): "Estimating DSGE models: Recent advances and future challenges," *Annual Review of Economics*, 13.

Fernández-Villaverde, J., S. Hurtado, and G. Nuño (2023): "Financial frictions and the wealth distribution," *Econometrica*, 91, 869–901.

Fernández-Villaverde, J., J. Marbet, G. Nuño, and O. Rachedi (2024): "Inequality and the zero lower bound," *Journal of Econometrics*, 105819.

Fernández-Villaverde, J., G. Nu no, G. Sorg-Langhans, and M. Vogler (2020): "Solving high-dimensional dynamic programming problems using deep learning," Manuscript.

Fernández-Villaverde, J., J. F. Rubio-Ramírez, and F. Schorfheide (2016): "Solution and estimation methods for DSGE models," in *Handbook of Macroeconomics*, Elsevier, vol. 2, 527–724.

Fernández-Villaverde, J. and D. Z. Valencia (2018): "A practical guide to parallelization in economics," Working Paper 24561, National Bureau of Economic Research.

Fershtman, C. and A. Pakes (2012): "Dynamic games with asymmetric information: A framework for empirical work," *Quarterly Journal of Economics*, 127, 1611–1661.

Folini, D., A. Friedl, F. Kübler, and S. Scheidegger (2024): "The climate in climate economics," *Review of Economic Studies*, rdae011.

Friedl, A., F. Kübler, S. Scheidegger, and T. Usui (2023): "Deep uncertainty quantification: With an application to integrated assessment models," Working paper, University of Lausanne.

Fudenberg, D. and D. K. Levine (1993): "Self-confirming equilibrium," *Econometrica*, 523–545.

Goodfellow, I. J., J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio (2014): "Generative adversarial networks," Tech. Rep. 1406.2661, arXiv.

Gopalakrishna, G. (2021): "ALIENs and continuous time economies," Swiss Finance Institute Research Paper Series 21-34, Swiss Finance Institute.

Gopalakrishna, G., Z. Gu, and J. Payne (2024): "Institutional asset pricing, segmentation, and inequality," Manuscript.

Gorodnichenko, Y., S. Maliar, and C. Naubert (2020): "Household savings and monetary policy under individual and aggregate stochastic volatility," Discussion paper 15614, CEPR.

Gu, Z., M. Lauriere, S. Merkel, and J. Payne (2023): "Global solutions to master equations for continuous time heterogeneous agent macroeconomic models," Available at SSRN 4871228.

Han, J., A. Jentzen, and W. E (2018): "Solving high-dimensional partial differential equations using deep learning," *Proceedings of the National Academy of Sciences*, 115, 8505–8510.

Han, J., Y. Yang, and W. E (2022): "DeepHAM: A global solution method for heterogeneous agent models with aggregate shocks," Tech. Rep. 2112.14377, arXiv.

Hansen, L. P. and T. J. Sargent (1980): "Formulating and estimating dynamic linear rational expectations models," *Journal of Economic Dynamics and Control*, 2, 7–46.

Hinterlang, N. and A. Tänzer (2021): "Optimal monetary policy using reinforcement learning," Discussion Papers 51/2021, Deutsche Bundesbank.

Hornik, K., M. Stinchcombe, and H. White (1989): "Multilayer feedforward networks are universal approximators," *Neural Networks*, 2, 359–366.

Huang, J. (2023): "Breaking the curse of dimensionality in heterogeneous-agent models: A deep learning-based probabilistic approach," Available at SSRN 4649043.

Jacot, A., S. H. Choi, and Y. Wen (2025): "How DNNs break the Curse of Dimensionality: Compositionality and Symmetry Learning," Tech. Rep. 2407.05664, arXiv.

Jumper, J. M., R. Evans, A. Pritzel, T. Green, M. Figurnov, O. Ronneberger, K. Tunyasuvunakool, R. Bates, A. Žídek, A. Potapenko, A. Bridgland, C. Meyer, S. A. A. Kohl, A. Ballard, A. Cowie, B. Romera-Paredes, S. Nikolov, R. Jain, J. Adler, T. Back, S. Petersen, D. Reiman, E. Clancy, M. Zielinski, M. Steinegger, M. Pacholska, T. Berghammer, S. Bodenstein, D. Silver, O. Vinyals, A. W. Senior, K. Kavukcuoglu, P. Kohli, and D. Hassabis (2021): "Highly accurate protein structure prediction with AlphaFold," *Nature*, 596, 583 – 589.

Kaji, T., E. Manresa, and G. Pouliot (2023): "An adversarial approach to structural estimation," *Econometrica*, 91, 2041–2063.

Kase, H., L. Melosi, and M. Rottner (2024): "Estimating nonlinear heterogeneous agent models with neural networks," Research Paper Series 1499, University of Warwick, Department of Economics.

Kelly, B. T. and D. Xiu (2023): "Financial machine learning," NBER Working Papers 31502, National Bureau of Economic Research.

Krizhevsky, A., I. Sutskever, and G. E. Hinton (2012): "Imagenet classification with deep convolutional neural networks," in *Advances in Neural Information Processing Systems*, 1097–1105.

Krusell, P. and A. A. Smith, Jr (1998): "Income and wealth heterogeneity in the macroeconomy," *Journal of Political Economy*, 106, 867–896.

Ledoux, M. (2001): *The Concentration of Measure Phenomenon*, American Mathematical Society.

Maliar, L. and S. Maliar (2003): "Parameterized expectations algorithm and the moving bounds," *Journal of Business & Economic Statistics*, 21, 88–92.

Maliar, L., S. Maliar, and P. Winant (2021): "Deep learning for solving dynamic economic models." *Journal of Monetary Economics*, 122, 76–101.

Mnih, V., K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis (2015): "Human-level control through deep reinforcement learning," *Nature*, 518, 529–533.

Moll, B. (2025): "The Trouble with Rational Expectations in Heterogeneous Agent Models: A Challenge for Macroeconomics," *Economic Journal*, ueaf104.

Murphy, K. (2025): "Reinforcement Learning: An Overview," Tech. Rep. 2412.05265, arXiv.

Murphy, K. P. (2022): *Probabilistic Machine Learning: An Introduction*, MIT Press.

——— (2024): *Probabilistic Machine Learning: Advanced Topics*, MIT Press.

Nagel, S. (2021): *Machine Learning in Asset Pricing*, Princeton University Press.

Naubert, C. (2025): "Differentiable, Filter Free Bayesian Estimation of DSGE Models Using Mixture Density Networks," Staff working papers, Bank of Canada.

Nuño, S. Scheidegger, and P. Renner (2024): "Let bygones be bygones: Optimal monetary policy with persistent supply shocks," Manuscript.

Payne, J., A. Rebei, and Y. Yang (2024): "Deep learning for search and matching models," Manuscript.

Prince, S. J. (2023): *Understanding Deep Learning*, MIT Press.

Rosenblatt, F. (1958): "The perceptron: A probabilistic model for information storage and organization in the brain." *Psychological Review*, 65, 386–408.

Sargent, T. J. (2025): "Macroeconomics after Lucas," *Journal of Political Economy*, 133, 3390–3417.

Sauzet, M. (2021): "Projection methods via neural networks for continuous-time models," Manuscript.

Schaeffer, R., M. Khona, Z. Robertson, A. Boopathy, K. Pistunova, J. W. Rocks, I. R. Fiete, and O. Koyejo (2023): "Double descent demystified: Identifying, interpreting and ablating the sources of a deep learning puzzle," Tech. Rep. 2303.14151, arXiv.

Scheidegger, S. and I. Bilionis (2019): "Machine learning for high-dimensional dynamic stochastic economies," *Journal of Computational Science*, 33, 68–82.

Shen, Z., R. Zhang, M. Dell, B. Lee, J. Carlson, and W. Li (2021): "LayoutParser: A unified toolkit for deep learning based document image analysis," *International Conference on Document Analysis and Recognition*, 131–146.

Silver, D., A. Huang, C. J. Maddison, A. Guez, L. Sifre, G. van den Driessche, J. Schrittwieser, I. Antonoglou, V. Panneershelvam, M. Lanctot, S. Dieleman, D. Grewe, J. Nham, N. Kalchbrenner, I. Sutskever, T. Lillicrap, M. Leach, K. Kavukcuoglu, T. Graepel, and D. Hassabis (2016): "Mastering the game of Go with deep neural networks and tree search," *Nature*, 529, 484–489.

Sirignano, J. and K. Spiliopoulos (2018): "DGM: A deep learning algorithm for solving partial differential equations," *Journal of Computational Physics*, 375, 1339–1364.

Smith, S. L., B. Dherin, D. Barrett, and S. De (2021): "On the origin of implicit regularization in stochastic gradient descent," in *International Conference on Learning Representations*.

Spiess, J., G. Imbens, and A. Venugopal (2023): "Double and single descent in causal inference with an application to high-dimensional synthetic control," NBER Working Papers 31802, National Bureau of Economic Research.

Trinh, T., Y. T. Wu, Q. Le, H. He, and T. Luong (2024): "Solving olympiad geometry without human demonstrations," *Nature*, 625, 476–482.

Vallet, F., J.-G. Cailton, and P. Refregier (1989): "Linear and nonlinear extension of the pseudo-inverse solution for learning Boolean functions," *Europhysics Letters*, 9, 315.

Vaswani, A., N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin (2017): "Attention is all you need," in *Advances in Neural Information Processing Systems*, 5998–6008.

Vershynin, R. (2018): *High-Dimensional Probability: An Introduction with Applications in Data Science*, vol. 47, Cambridge University Press.

Villa, A. T. and V. Valaitis (2024): "Machine learning projection method for macro-finance models," *Quantitative Economics*, 15, 145–173.

Voth, H.-J. and D. Yanagizawa-Drott (2024): "Image(s)," Tech. rep., University of Zurich.

Weintraub, G. Y., C. L. Benkard, and B. Van Roy (2008): "Markov perfect industry dynamics with many firms," *Econometrica*, 76, 1375–1411.

——— (2010): "Computational methods for oblivious equilibrium," *Operations Research*, 58, 1247–1265.

Wilensky, U. and W. Rand (2015): *An Introduction to Agent-Based Modeling: Modeling Natural, Social, and Engineered Complex Systems with NetLogo*, MIT Press.

Wilson, A. G. (2025): "Deep learning is not so mysterious or different," in *Forty-second International Conference on Machine Learning Position Paper Track*.

Winberry, T. (2018): "A method for solving and estimating heterogeneous agent macro models," *Quantitative Economics*, 9, 1123–1151.

Zhuang, F., Z. Qi, K. Duan, D. Xi, Y. Zhu, H. Zhu, H. Xiong, and Q. He (2020): "A comprehensive survey on transfer learning," *Proceedings of the IEEE*, 109, 43–76.